

Звіт про тестування

Що тестували, що пройшло і що означають результати — включно з двома реальними помилками, які виявили тести

Зрозумілий виклад перевірки: Java-набір середовища виконання, Python-набір навчання, результати відокремлення без міток, дві приховані помилки, які виявили тести, та їх виправлення, і чесна заява про те, що ці результати доводять, а що ні.

Документ	Звіт про тестування
Продукт	Jneopallium Self-Supervised Maintenance Guardian
Java-набір	SelfSupervisedMaintenanceModuleTest — 14/14 успішно
Python-набір	tests/test_ss_maintenance.py — 5/5 успішно
Метод	Детермінований; мітки відмов лише для оцінки, ніколи для навчання
Автор	Дмитро Раковський — Харків, Україна
Дата	2 липня 2026
Ліцензія	BSD 3-Clause

Зміст

1. Підсумок
2. Для нефахівця: як ми тестували без міток
3. Що було під тестом
4. Java-набір середовища виконання (14 тестів)
5. Python-набір навчання (5 тестів)
6. Результати відокремлення без міток
7. Дві помилки, які виявили тести
8. Детермінізм і відтворюваність
9. Що ці результати доводять — а що ні
10. Як відтворити

1 Підсумок

Набір	Тести	Результат
Java-середовище (SelfSupervisedMaintenanceModule Test)	14	усі успішно
Python-навчання (test_ss_maintenance)	5	усі успішно
Знайдені й виправлені приховані помилки	2	переповнення в дедуп і обмеженні частоти

Головне

Кожне впроваджене сімейство несправностей виявляється без жодної мітки, здорові активи мовчать, цикл зворотного зв'язку рухає пороги у правильний бік, і модель доказово не може керувати. Дві реальні помилки переповнення міток часу знайдено тестами й виправлено.

2 Для нефахівця: як ми тестували без міток

Керовану модель тестувати легко: ховаєш кілька розмічених відмов і перевіряєш, чи вона їх знаходить. У цієї моделі міток немає, тож тест інший. Ми генеруємо реалістичну телеметрію, потайки впроваджуємо повільну деградацію в деякі активи й тримаємо час цієї деградації в запечатаному «ключі відповідей», якого модель не бачить. Модель навчається лише на здорових даних. Потім перевіряємо: чи позначила вона активи, що деградують (використовуючи ключ лише для оцінки), і чи лишила здорові в спокої? Це чесний тест детектора без міток.

3 Що було під тестом

- **Середовище (Java):** чотири нейрони, шість сигналів і п'ять процесорів — модель, що справді працює у продакшені.
- **Навчання (Python):** припасування без міток і розгортувана збірка, яку воно генерує.
- **Інваріанти безпеки:** що модель рекомендаційна й не може бути перемкнена на керування.

4 Java-набір середовища виконання (14 тестів)

Виконано через справжню платформу JUnit. Покриття за областями:

Область	Що стверджується
Відновлення	здоровий кадр → малий залишок; аномалія → великий залишок на правильному сенсорі; позначено зсув домену
Гіпотеза	зростаючий залишок накопичує свідчення, підіймає тяжкість, приписує сімейство й дає запас часу; здоровий потік мовчить
Зворотний зв'язок	хибне спрацювання підіймає поріг, підтвердження послаблює, адаптація замерзає під час зсуву, перше

	оновлення не придушується
Рекомендаційний шлюз	спрацьовує за порогом, дедуплікує, застосовує живі оновлення порога, перша рекомендація не придушується, <code>advisoryOnly true</code>
Конфіг / процесори	інваріант «лише рекомендація» тримається; усі процесори типізовані через інтерфейс

5 Python-набір навчання (5 тестів)

Тест	Що підтверджує
перцентилі впорядковані	<code>mean < p95 < p99 < p999</code>
ваги скінченні	кожна припасована вага — скінченне число
здорові кадри — малий залишок	здоровий актив перевищує власний p99 менш ніж у 5% випадків
несправності відокремлюються без міток	кожне сімейство понад 2× p999 і вказує на правильну групу сенсорів
збірка записана й валідна	<code>labelFree=true, labelsUsed=0, safetyMode=ADVISORY</code> , шари несуть припасовані ваги

6 Результати відокремлення без міток

Здоровий базовий рівень (припасований лише на здорових даних): `mean` 0.78, `p99` 2.08, `p999` 2.75 в одиницях помилки відновлення. Відносно цього базового рівня кожна впроваджена деградація — знос підшипника, кавітація, дрейф сенсора, погіршення енергоефективності та коливання контуру — піднімається понад **вдвічі** вище екстремуму `p999` на повному розгоні, і домінантний залишок визначає правильну групу сенсорів у кожному випадку. Здорові активи перевищують власний `p99` менш ніж у 5% випадків, а вимога стійкості в накопичувачі свідчень придушує більшість цих перехідних процесів перш ніж вони стануть рекомендаціями.

Чому це важливо

Це показує, що ядро без міток справді відокремлює несправності від норми, спираючись лише на зв'язок між сенсорами — історія відмов не залучалася жодного разу.

7 Дві помилки, які виявили тести

Димові й модульні тести виявили два справжні приховані дефекти, обидва виправлено й тепер захищено від регресії:

- **Переповнення дедупу шлюзу.** Дедуплікація за активом використовувала сентинел `Long.MIN_VALUE; timestamp - sentinel` переповнюється на реальних мітках часу в мілісекундах, що хибно придушувало **першу** рекомендацію для кожного активу.
- **Переповнення обмеження частоти зворотного зв'язку.** Той самий шаблон сентинела в адаптері проковтнув би **перше** оновлення зворотного зв'язку для кожного сімейства у продакшені.

Обидва тепер є перевіркою на null замість віднімання сентинела, з окремими тестами («перша рекомендація не придушується», «перша подія не придушується»), що використовують реалістичні мітки часу епохи.

8 Детермінізм і відтворюваність

Навчання й синтетичний корпус мають фіксоване зерно, тож результати відтворюються біт у біт. Час несправностей живе лише в оракулі оцінювання; він ніколи не входить у припасування. Обидва набори працюють офлайн без сторонніх Python-залежностей.

9 Що ці результати доводять — а що ні

- **Вони доводять**, що логіка середовища виконання коректна, інваріант безпеки тримається, а припасування без міток відокремлює змодельовані сімейства несправностей на реалістичних синтетичних даних.
- **Вони не доводять** польову точність на вашій установці. Відокремлення на синтетичному корпусі — це не польова частка виявлення; реальні активи безладніші, а сімейства несправностей евристичні, доки не накопичаться підтверджені події.
- **Тому:** спершу розгортайте в тіні, вимірюйте частку хибних спрацювань на своїх даних і дайте циклу зворотного зв'язку відкалібрувати — саме така розкатка, що приписує посібник з розгортання.

10 Як відтворити

```
# python-набір навчання
cd scripts/demo-self-supervised-maintenance
python -m unittest tests.test_ss_maintenance -v

# java-набір середовища виконання
mvn -pl worker -Dtest=SelfSupervisedMaintenanceModuleTest test
```

Тести тримають модель на чесній планці: виявляти несправності без міток, ніколи не керувати й вдосконалюватися зі зворотного зв'язку — і вони виправдали себе, зловивши дві реальні помилки до продакшену.